

Pipelined ALU Project Architecture and Dataflow Description

Introduction

This document explains the complete architecture and dataflow of the pipelined ALU project. The design is implemented in Verilog using a modular RTL approach and is organized into separate blocks such as the **control unit, register file, pipeline registers, ALU core, and top-level integration module**. The project follows an industry-style synchronous hardware design methodology, where instruction fields, operand values, destination addresses, and status flags move through the pipeline in a coordinated and timing-aware manner.

Project Objective

The objective of this project is to build a synthesizable pipelined arithmetic logic unit that can execute multiple operations in a structured and verifiable manner. The design supports arithmetic, logical, shift, inversion, buffer, and comparison operations. The pipeline is used to improve throughput and to separate instruction decoding, operand fetch, execution, and writeback into distinct timing stages.

Top-Level Architecture Overview

At the top level, the design instantiates the control unit, register file, pipeline registers, ALU module, and output multiplexing logic. The top-level module acts as the integration point for all submodules. It receives the external instruction, input data, valid and load control signals, and clock/reset inputs. The final result and status flags are observed from this top-level module.

The architecture can be understood as two cooperating parts. The first part is the control path, which decodes the instruction and produces addresses, opcode information, write enable, and other control signals. The second part is the datapath, which stores operands, performs ALU operations, propagates data through pipeline registers, and writes results back to the register file.

Instruction Format and Opcode Usage

The instruction word is a compact control packet that carries the operation code and the register addressing information needed for execution. The opcode selects the ALU function, while the source and destination addresses identify where operands are read from and where the result is written.

The supported opcode table includes buffer operations, arithmetic operations such as addition and subtraction, logical operations such as AND, OR, and XOR, bitwise inversion, shift operations, and compare functionality. Each opcode is decoded by the control unit and then forwarded into the pipeline so that the ALU performs the correct operation at the execute stage.

Control Path Description

The control unit is responsible for decoding the incoming instruction and producing all required control signals. It determines the source register addresses, destination register address, opcode, write enable behavior, and clear/reset-related control outputs. The control unit does not perform arithmetic computation. Its role is to coordinate the movement of instruction information through the pipeline.

In the pipelined system, the control information must remain aligned with the data. This means the opcode, destination address, and write control must travel through matching pipeline stages so that the correct result is written into the correct register at the correct cycle.

Datapath Description

The datapath carries the actual operand values and computed results. The register file provides the operand values required by the ALU. The pipeline registers hold intermediate values between stages. The ALU performs the selected operation on the input operands and generates the output result along with status flags.

The datapath is fully synchronous. Each pipeline register captures its input on the active clock edge and presents a stable output during the next cycle. This behavior ensures that each instruction advances through the design in an orderly and predictable manner.

Pipeline Stage Breakdown

Stage 1 is the instruction decode stage. The instruction is accepted, decoded, and translated into opcode and register address information. The control unit also prepares the control bits required for later stages.

Stage 2 is the register read stage. The register file uses the source addresses to fetch operand A and operand B. These operand values are then stored in pipeline registers so that they can be supplied to the ALU at the next stage.

Stage 3 is the execute stage. The ALU receives the operands and opcode and computes the requested operation. It also generates the carry, overflow, zero, and negative flags according to the selected operation.

Stage 4 is the writeback stage. The computed result is selected by the output multiplexer and written back into the destination register of the register file when write enable is active.

Role of Pipeline Registers

Pipeline registers are the most important timing elements in the design. They divide the datapath into separate stages and ensure that each stage sees a stable input for one full clock cycle. Without pipeline registers, the combinational delay of the complete instruction path would be too large and the design would be harder to time and verify.

Each pipeline register stores the information required by the next stage. For example, the decode-to-read register stores addresses and opcode information, the read-to-execute register stores operand values along with the opcode, and the execute-to-writeback register stores the final result, and destination address.

The pipeline registers also preserve instruction ordering. Multiple instructions may be inside the pipeline at the same time, but each instruction carries its own control and data context through the stage registers.

Register File Interaction

The register file acts as the storage bank for operands and results. It supports two read ports so that two source operands can be read simultaneously, and one write port so that the ALU result can be written back into the selected destination register.

During execution, the source register addresses from the control unit select the operands. After the ALU finishes the computation and the pipeline reaches the writeback stage, the write enable signal and destination address determine where the result will be stored.

ALU Operation Flow

The ALU receives two 32-bit operands and a 4-bit opcode. The opcode determines whether the ALU should perform buffering (passing value), addition, subtraction, shifts, logical operations, inversion, or comparison. The operation result is produced as a 32-bit output.

For arithmetic operations, the ALU also calculates status flags. For logical and shift operations, the flags are generated according to the chosen implementation rules and the resulting output value.

Status Flag Generation

The project uses four main status flags: carry, overflow, zero, and negative. These are generated as combinational outputs of the ALU stage and are associated with the current instruction being executed.

The carry flag indicates unsigned carry or borrow behavior, depending on the operation. The overflow flag indicates signed arithmetic overflow for operations where overflow is meaningful. The zero flag indicates that the final ALU result is equal to zero. The negative flag reflects the sign bit of the result in two's complement representation.

Output Selection and Writeback

The output multiplexer selects whether the final writeback data comes from the ALU result or from the external input path, depending on the load control signal. This allows the design to support both direct loading of external data and writeback of computed ALU results.

At the writeback stage, the selected data is written into the destination register when write enable is asserted. This final step completes the execution of the instruction.

Example Dataflow Sequence

Example sequence:

1. An instruction enters the control unit.
2. The opcode and addresses are decoded.
3. Source operands are read from the register file.
4. The operands move through pipeline registers.
5. The ALU performs the requested operation.
6. Flags are generated along with the result.
7. The result is selected by the output mux.
8. The destination register is written at the writeback stage.

Reset and Valid Behavior

The design includes reset and valid-related control behavior to keep the pipeline in a known state. Reset initializes the sequential elements, while valid control ensures that only meaningful instructions are propagated through the datapath.

These signals are important during simulation, startup, and when testing instruction sequences. They help avoid undefined data movement and improve verification clarity.

Design Assumptions and Limitations

The design assumes that instructions are supplied in a controlled manner and that the pipeline stages are synchronized by the system clock. The project focuses on clean RTL implementation and verification rather than on advanced processor features such as hazard resolution, forwarding, or branching.

Verification Summary

The architecture has been verified using module-level testbenches and a top-level SystemVerilog testbench. The testbench checks opcode decoding, operand movement, pipeline register behavior, flag generation, register file writeback, and output correctness.

Conclusion

This project demonstrates a complete pipelined ALU architecture built using modular Verilog RTL. It shows how a control path and datapath can work together to process instructions in a pipeline, while maintaining correct alignment of operands, results, addresses, and flags. The design is suitable as a strong RTL portfolio project because it combines architecture, timing, modularity, and verification in a single implementation.